CS-401

Parallel and Distributed Computing

Lecture 4

Dr. Tariq Sadad

DCS, IIUI

Some General Parallel Terminology

- **Supercomputing / High Performance Computing (HPC)**
  - Using the world's fastest and largest computers to solve large problems
- **Node**
  - A standalone "computer in a box". Usually comprised of multiple CPUs/processors/cores, memory, network interfaces, etc. Nodes are networked together to comprise a supercomputer.
- **CPU / Socket / Processor / Core**
  - In the past, a CPU (Central Processing Unit) was a singular execution component
  - Then, multiple CPUs were incorporated into a node
  - Then, individual CPUs were subdivided into multiple "cores", each being a unique execution unit
  - CPUs with multiple cores are sometimes called "sockets"
  - The result is a node with multiple CPUs, each containing multiple cores

Memory Programming

- Shared Memory Programming
  - Pthreads
  - C++ Thread
  - OpenMP (Open Multi-Processing is comparatively better)

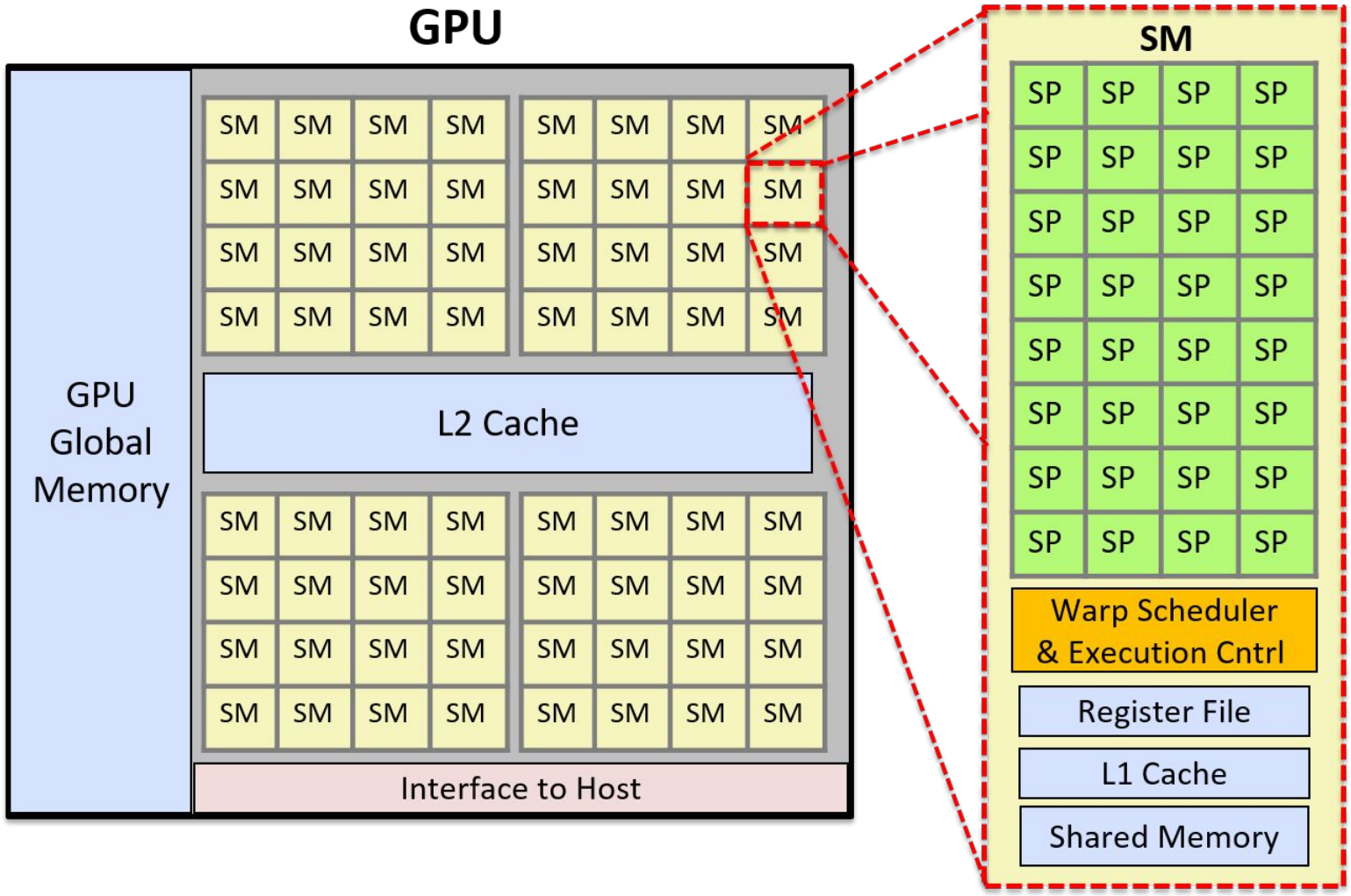
- Distributed Memory Programming
  - MPI (Message Passing Interface)
    - Different Methods: MPI_Init, MPI_send, MPI_Receive, MPI_Finalize
  - PGAS (Partitioned Global Address Space)

Accelerator Programming

- Hybrid programming for hybrid architecture.
  - Some portion of the program on CPU and remaining task execution on GPU.

- **GPU-accelerated Computing**
- The GPU programming are done inside CUDA or OpenCL
- GPU (Graphics Processing Units): for Computer graphics and Image Processing. E.g. game industry.
  - CUDA (Compute Unified Device Architecture) for graphics devices by NVIDIA
  - OpenCL
- GPU has thousand special purpose processors. Process pixels in a 2D image in parallel. It implements SIMD model. E.g. NVIDIA GPUs consist of several streaming multiprocessors (SMs), each of which has its own execution control units and memory space (registers, L1 cache, and shared memory). Each SM consists of several scalar processor (SP) cores.

Graphics Processing Unit (GPU)



GPU-accelerated Computing

- GPU-accelerated computing is the employment of a graphics processing unit (GPU) along with a computer processing unit (CPU) in order to facilitate processing-intensive operations such as deep learning, analytics and engineering applications.
- CPU is comprised of cores designed for sequential serial processing, the GPU is designed with a parallel architecture consisting of more efficient yet smaller cores that can easily handle multiple tasks in parallel.
- GPU-accelerated computing deployment is growing in popularity due to the large variety of applications in which it could be used, such as video editing, medical imaging, Machine Learning and artificial intelligence, drones, robots or autonomic cars, IoT computing, .

- Process vs Thread
  - Applications consists of one or more processes.
  - Both processes and threads are independent sequence of executions.
  - Thread (of the same process) run in a shared memory space, while processes run in separate memory spaces.

- Concurrency vs Parallelism
  - Concurrency is when two or more tasks can start, run, and complete is overlapping time periods. It doesn’t necessarily mean they will ever both be running at the same instant. For example, multitasking on a single-core machine.
  - Parallelism is when tasks literally run at the same time, e.g. on a multicore processor.

Intel Hyper-Threading (HT)

- Intel Hyper-Threading (HT) is a technology introduced by Intel in some of its CPU architectures.
- HT enables a single physical CPU core to behave as if it were two logical cores, allowing it to execute multiple threads simultaneously.
- Each physical core with HT appears as two logical cores to the operating system, so a dual-core CPU with HT would appear as if it has four logical cores.
- HT does not add physical processing cores but rather enhances the efficiency of a single core by allowing it to overlap the execution of multiple threads.
- HT can improve the overall throughput of a CPU when executing multiple tasks that can be parallelized.
- HT is primarily aimed at improving single-core performance and is most effective when the workload has multiple threads that can be executed concurrently.

Multithreading

- Multithreading is a broader concept that applies to the execution of multiple threads or tasks in parallel on a CPU or within a software program.
- Multithreading can be implemented on CPUs with or without HT technology.
- In a multithreaded application, multiple threads are created to perform different tasks simultaneously, such as in a multi-threaded software program.
- Multithreading can be implemented using multiple CPU cores (physical or logical) or even on a single core without HT.
- It is a software or programming technique to take advantage of parallelism and is not limited to Intel HT.
- Multithreading can improve the responsiveness and performance of applications by effectively utilizing the available CPU resources

- FLOPS: To measure the computational power. Theoretical peak performance.

- Floating point operations per second
- Measure of theoretical peak performance

$$\mathbf{FLOPS} = \mathbf{sockets} \times \frac{\mathbf{cores}}{\mathbf{socket}} \times \frac{\mathbf{cycles}}{\mathbf{second}} \times \frac{\mathbf{FLOPs}}{\mathbf{cycle}}$$

FLOPS

- Servers are the only computers that sometimes have more than one socket; for most home computers (desktop or laptop), “sockets” will be 1.
- Cores per socket depends on your CPU. It could be 2 (dual-core), 3, 4 (quad-core), 6 (hexa-core), or 8. There are some prototype CPUs with as many as 80 cores.
- “Clock cycles per second” refers to the speed of your CPU. Most modern CPUs are rated in gigahertz. So 2 GHz would be 2,000,000,000 clock cycles per second.
- The number of FLOPs per cycle also depends on the CPU. One of the fastest (home computer) CPUs is the Intel Core i7–970, capable of 4 double-precision or 8 single-precision floating-point operations per cycle.

Test

- Intel Core i7–970 has 6 cores. If it is running at 3.46 GHz and can perform 8 floating point operations per second, calculate the theoretical compute power of this machine.

- Intel Core i7–970 has 6 cores. If it is running at 3.46 GHz, the formula would be:
- 1 (socket) * 6 (cores) * 3,460,000,000 (cycles per second) * 8 (single-precision FLOPs per second) = 166,080,000,000 single-precision FLOPs per second or 83,040,000,000 double-precision FLOPs per second.
- 109 GFLOPS.

Speedup Calculation: Amdels' Law

Amdahl's Law is a principle used to calculate the maximum possible speedup of a program when a portion of it is parallelized.

Formula: $\text{Speedup} = \frac{1}{(1-P)+\frac{P}{N}}$

Where:

- **P** is the fraction of the program that can be parallelized.
- **N** is the number of processors (or parallel processing units) used.
- **1 - P** is the fraction of the program that must be executed serially (i.e., cannot be parallelized).

Example Calculation

Assume a program where:

- 80% (P = 0.80) of the program can be parallelized.
- 20% (1 - P = 0.20) of the program must be executed serially.
- We have 4 processors (N = 4).

Applying Amdahl's Law: $\text{Speedup} = \frac{1}{(1-0.80)+\frac{0.80}{4}}$ $\text{Speedup} = \frac{1}{0.20+0.20}$ $\text{Speedup} = \frac{1}{0.40}$

$\text{Speedup} = 2.5$

In this example, the theoretical maximum speedup of the program with 4 processors is 2.5 times faster than running it on a single processor.

Challenges in Parallelism

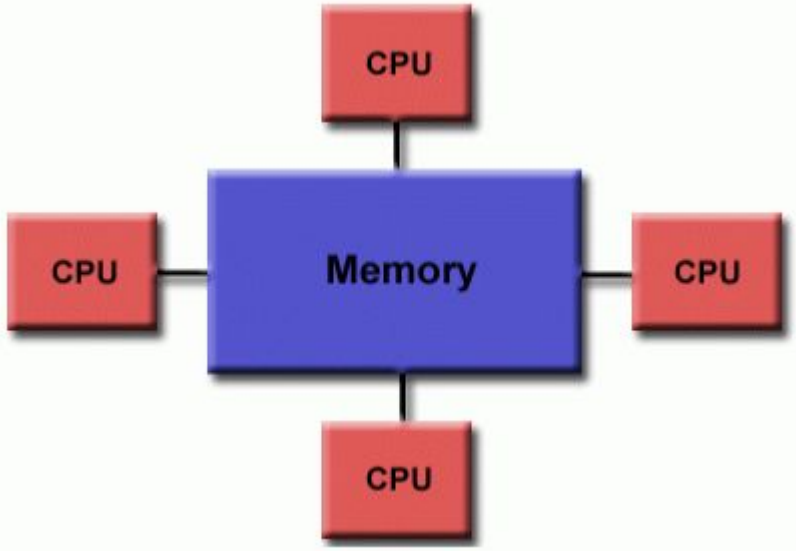
- **Complexity**
- In general, parallel applications are more complex than corresponding serial applications. Not only do you have multiple instruction streams executing at the same time, but you also have data flowing between them.
- The costs of complexity are measured in programmer time in virtually every aspect of the software development cycle:
  - Design
  - Coding
  - Debugging
  - Tuning
  - Maintenance
- Adhering to "good" software development practices is essential when developing parallel applications.

- **Portability**
- Thanks to standardization in several APIs, such as MPI, OpenMP and POSIX threads, portability issues with parallel programs are not as serious as in years past. However,
- All of the usual portability issues associated with serial programs apply to parallel programs. For example, if you use vendor "enhancements" to Fortran, C or C++, portability will be a problem.
- Even though standards exist for several APIs, implementations will differ in a number of details, sometimes to the point of requiring code modifications in order to effect portability.
- Operating systems can play a key role in code portability issues.
- Hardware architectures are characteristically highly variable and can affect portability.

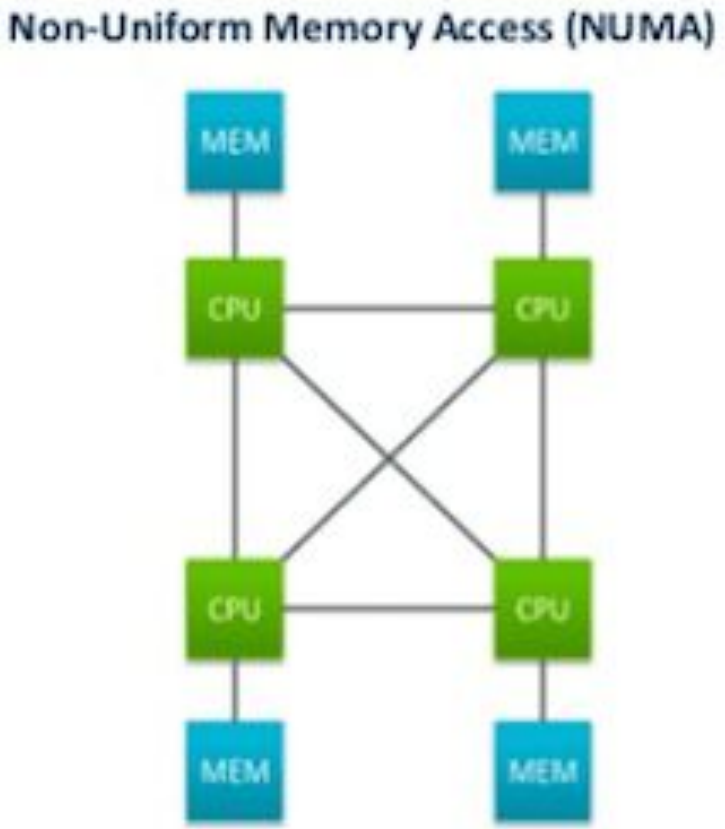
- **Resource Requirements**
- The primary intent of parallel programming is to decrease execution wall clock time, however in order to accomplish this, more CPU time is required. For example, a parallel code that runs in 1 hour on 8 processors actually uses 8 hours of CPU time.
- The amount of memory required can be greater for parallel codes than serial codes, due to the need to replicate data and for overheads associated with parallel support libraries and subsystems.
- For short running parallel programs, there can actually be a decrease in performance compared to a similar serial implementation. The overhead costs associated with setting up the parallel environment, task creation, communications and task termination can comprise a significant portion of the total execution time for short runs.

Parallel Computer Memory Architectures

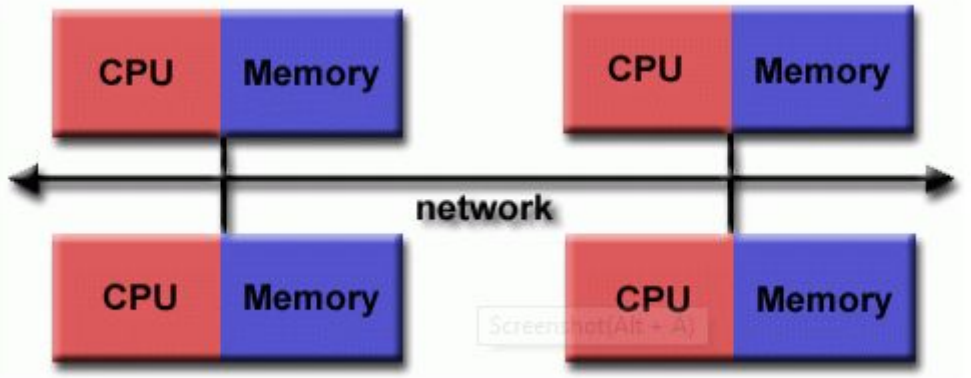
- **Uniform Memory Access (UMA)**
  - Most commonly represented today by ***Symmetric Multiprocessor (SMP)*** machines
  - Identical processors
  - Equal access and access times to memory
  - Sometimes called CC-UMA - Cache Coherent UMA. Cache coherent means if one processor updates a location in shared memory, all the other processors know about the update. Cache coherency is accomplished at the hardware level.



- **Non-Uniform Memory Access (NUMA)**
  - Often made by physically linking two or more SMPs
  - One SMP can directly access memory of another SMP
  - Not all processors have equal access time to all memories
  - Memory access across link is slower
  - If cache coherency is maintained, then may also be called CC-NUMA - Cache Coherent NUMA



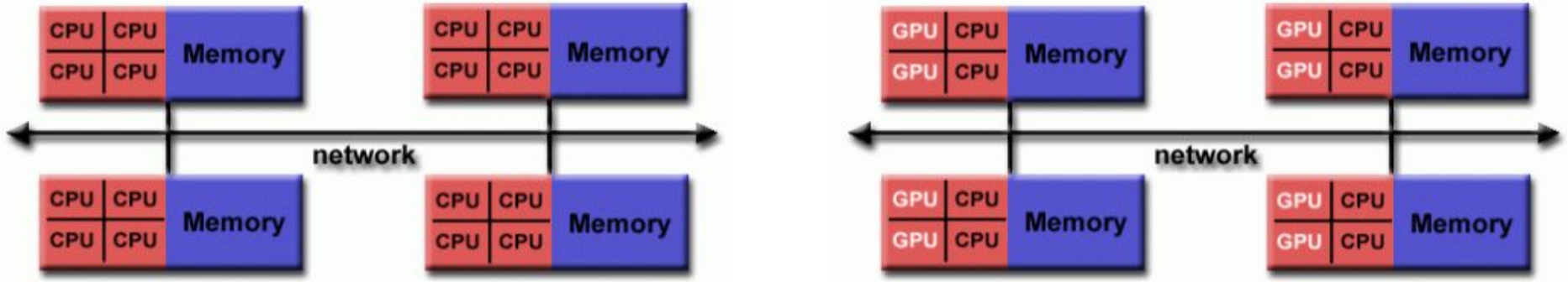
Distributed Memory Architecture



- Like shared memory systems, distributed memory systems vary widely but share a common characteristic. Distributed memory systems require a communication network to connect inter-processor memory.
- Processors have their own local memory. Memory addresses in one processor do not map to another processor, so there is no concept of global address space across all processors.
- Because each processor has its own local memory, it operates independently. Changes it makes to its local memory have no effect on the memory of other processors. Hence, the concept of cache coherency does not apply.
- When a processor needs access to data in another processor, it is usually the task of the programmer to explicitly define how and when data is communicated. Synchronization between tasks is likewise the programmer's responsibility.

Hybrid Distributed-Shared Memory

- The largest and fastest computers in the world today employ both shared and distributed memory architectures.
- The shared memory component can be a shared memory machine and/or graphics processing units (GPU).
- The distributed memory component is the networking of multiple shared memory/GPU machines, which know only about their own memory - not the memory on another machine. Therefore, network communications are required to move data from one machine to another.
- Current trends seem to indicate that this type of memory architecture will continue to prevail and increase at the high end of computing for the predictable future.



OpenMP: Open Multi-Processing

- An Application Program Interface (API) that may be used to explicitly direct ***multi-threaded, shared memory parallelism***
- Comprised of three primary API components:
  - Compiler Directives
  - Runtime Library Routines
  - Environment Variables
- First attempt at a standard was the draft for ANSI X3H5 in 1994..
- The OpenMP standard specification started in the spring of 1997.
- Initially, the API specifications were released separately for C and Fortran. Since 2005, they have been released together.
- The table below chronicles the OpenMP API release history:

Month/Year	Version
Oct 1997	Fortran 1.0
Oct 1998	C/C++ 1.0
Nov 1999	Fortran 1.1
Nov 2000	Fortran 2.0
Mar 2002	C/C++ 2.0
May 2005	OpenMP 2.5
May 2008	OpenMP 3.0
Jul 2011	OpenMP 3.1
Jul 2013	OpenMP 4.0
Nov 2018	OpenMP 5.0
Nov 2020	OpenMP 5.1
Nov 2021	OpenMP 5.2

OpenMP: Introduction

- OpenMP is designed for multi-processor/core, shared memory machines. The underlying architecture can be shared memory UMA or NUMA.

OpenMP Execution Model

- **Thread Based Parallelism (1/2):**
- OpenMP programs accomplish parallelism exclusively through the use of threads.
- A thread of execution is the smallest unit of processing that can be scheduled by an operating system. The idea of a subroutine that can be scheduled to run autonomously might help explain what a thread is.
- Threads exist within the resources of a single process. Without the process, they cease to exist.
- Typically, the number of threads match the number of machine processors/cores. However, the actual use of threads is up to the application.

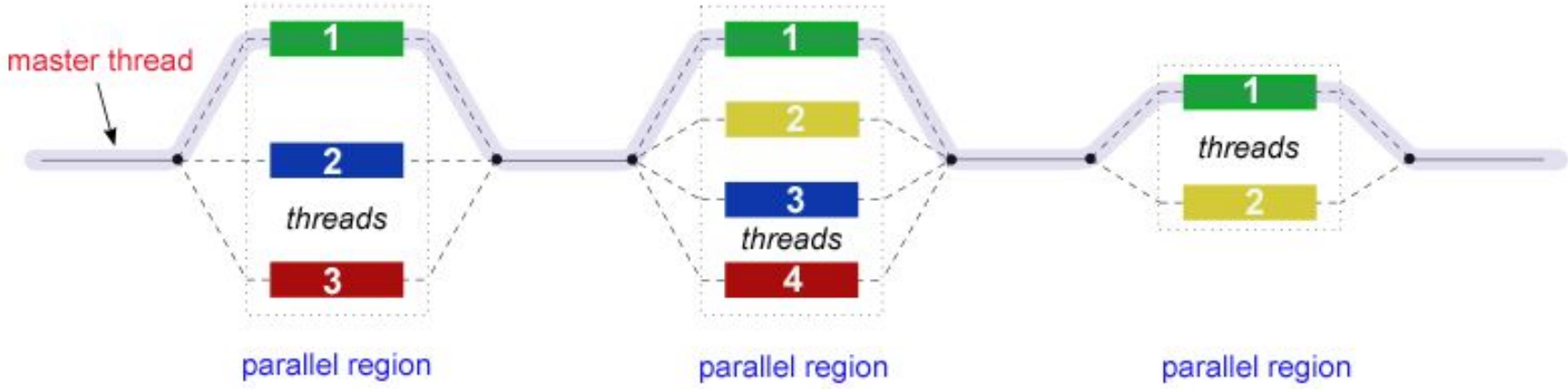
- **Thread Based Parallelism (2/2)**
- Multithreading is a CPU (central processing unit) feature that allows two or more instruction threads to execute independently while sharing the same process resources.
- A thread is a self-contained sequence of instructions that can execute in parallel with other threads that are part of the same root process.
- Multithreading allows multiple concurrent tasks can be performed within a single process.

OpenMP Execution Model

- **Explicit Parallelism:**
- OpenMP is an explicit (not automatic) programming model, offering the programmer full control over parallelization.
- Parallelization can be as simple as taking a serial program and inserting compiler directives….
- Or as complex as inserting subroutines to set multiple levels of parallelism, locks and even nested locks.

OpenMP Execution Model

- **Fork - Join Model:**
- OpenMP uses the fork-join model of parallel execution:



- All OpenMP programs begin as a single process: the **master thread**. The master thread executes sequentially until the first **parallel region** construct is encountered.
- **FORK**: the master thread then creates a team of parallel *threads*.
- The statements in the program that are enclosed by the parallel region construct are then executed in parallel among the various team threads.
- **JOIN**: When the team threads complete the statements in the parallel region construct, they synchronize and terminate, leaving only the master thread.
- The number of parallel regions and the threads that comprise them are arbitrary.

OpenMP Execution Model

- **Compiler Directive Based:**
  - Most OpenMP parallelism is specified through the use of compiler directives which are imbedded in C/C++.
- **Nested Parallelism:**
  - The API provides for the placement of parallel regions inside other parallel regions.
  - Implementations may or may not support this feature.
- **Dynamic Threads:**
  - The API provides for the runtime environment to dynamically alter the number of threads used to execute parallel regions. Intended to promote more efficient use of resources, if possible.
  - Implementations may or may not support this feature.
- **I/O:**
  - OpenMP specifies nothing about parallel I/O.
  - It is entirely up to the programmer to ensure that I/O is conducted correctly within the context of a multi-threaded program.

OpenMP Execution Model

- **Execution Model and Memory Model Interactions:**
  - Single-Program-Multiple-Data (SPMD) is underlying programming paradigm - all threads have potential to execute the same program code, however, each thread may access modify different data and traverse different execution paths

- **OpenMP Programming:**
  - Method to start up parallel threads
  - Method to discover how many threads are running
  - Need way to uniquely identify threads
  - Method to join threads for serial execution
  - Method to synchronize threads
  - Ensure consistent view of data items when necessary
  - Required to check for data dependencies, data conflicts, race conditions, or deadlocks

OpenMP API Overview

- Read MS-Word document.

- **OpenMP 4.0 Support:**
  - GNU: 4.9 for C/C++.